EECS 388

Introduction to Computer Security

Lecture 5:

Combining Confidentiality and Integrity

September 15, 2026
Prof. Halderman



Review: Stream Cipher

More practical approach to confidentiality:

Use a **pseudorandom generator (PRG)** instead of a truly random pad

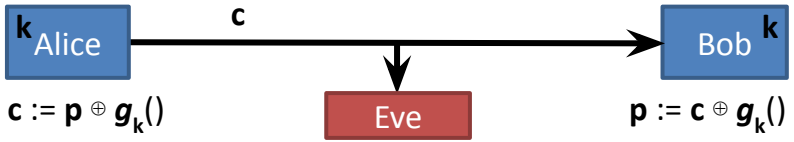
Recall: A PRG $g_k()$ is practically indistinguishable from a random stream of bits, unless you know **k**.

Called a **stream cipher**

Alice and Bob choose PRG $g()$, share secret key **k**

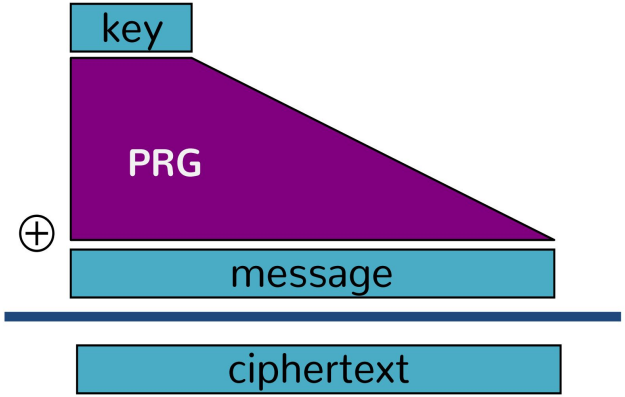
Encryption: $s := g_k();\ c_i := p_i \oplus s_i$

Decryption: $s := g_k();\ p_i := c_i \oplus s_i$



Caution! **NEVER reuse keys**

NEVER reuse PRG output bits



Provably secure if $g()$ is a secure PRG, under complexity assumptions

However… we don't know how to prove that secure PRGs even exist!

Best we can do (again): Use well studied functions where we haven't spotted a problem yet

Examples: ~~**RC4**~~ **ChaCha20**

Review: Block Ciphers

Another approach:

Block ciphers

consist of a function that **encrypts** fixed-size (**n**-bit) blocks with a reusable key **k**:

$$E_{\mathbf{k}}(\mathbf{p}) : \{0,1\}^{|\mathbf{k}|} \times \{0,1\}^{\mathbf{n}} \rightarrow \{0,1\}^{\mathbf{n}}$$

and an inverse function that **decrypts** the blocks when used with same key:

$$D_{\mathbf{k}}(\mathbf{c}) = E_{\mathbf{k}}^{-1}(\mathbf{c}) : \{0,1\}^{|\mathbf{k}|} \times \{0,1\}^{\mathbf{n}} \rightarrow \{0,1\}^{\mathbf{n}}$$

such that $\forall \mathbf{k} : D_{\mathbf{k}}(E_{\mathbf{k}}(\mathbf{p})) = \mathbf{p}$.

In effect, **k** selects one *permutation* from the set of $2^n!$ possible permutations of E's domain.

A block cipher is ***different*** from a PRF.

What do we want, if not a PRF?

Pseudorandom permutation (PRP)

A secure PRP is a function that cannot practically be distinguished from a truly random permutation unless you know **k**.

Annoying question again:

Do PRPs *actually exist*?

Same annoying answer:

We don't know. :(

Best we can do:

Design a complex function that's invertible if and (hopefully) only if you know **k**

Examples: ~~**DES**~~ **AES**

Review: Cipher Modes

Cipher-block chaining (CBC) mode

"Chains" ciphertexts to obscure later ones

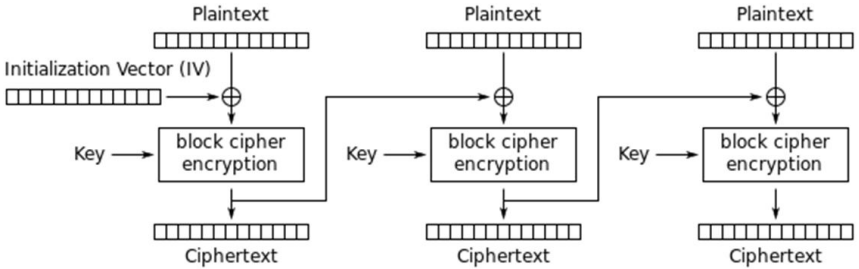
Choose a random **initialization vector IV**

Encrypt: $\mathbf{c}_0 := \mathbf{IV}$; $\mathbf{c}_i := \boldsymbol{E}_k(\mathbf{p}_i \oplus \mathbf{c}_{i-1})$

Decrypt: $\mathbf{p}_i := \boldsymbol{D}_k(\mathbf{c}_i) \oplus \mathbf{c}_{i-1}$,

Have to send IV with ciphertext

Can't encrypt blocks in parallel or out of order



Counter (CTR) mode

Turns a block cipher into a stream cipher

Generate **keystream s** for **k** and unique **nonce**:

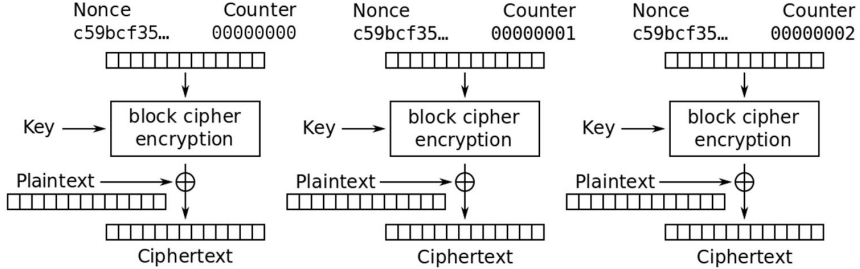
$\mathbf{s} := \boldsymbol{E}_k(\mathbf{nonce}\|0) \| \boldsymbol{E}_k(\mathbf{nonce}\|1) \| \boldsymbol{E}_k(\mathbf{nonce}\|2) \| \ldots$

Encrypt: $\mathbf{c} := \mathbf{p} \oplus \mathbf{s}$ Decrypt: $\mathbf{p} := \mathbf{c} \oplus \mathbf{s}$

Benefits: Doesn't require padding

Efficient parallelism/random access

Caution: Never reuse **nonce** for same **k**!



Getting both confidentiality and integrity?

Integrity (~~tampering~~)

Let ***f***() be a secure PRF.

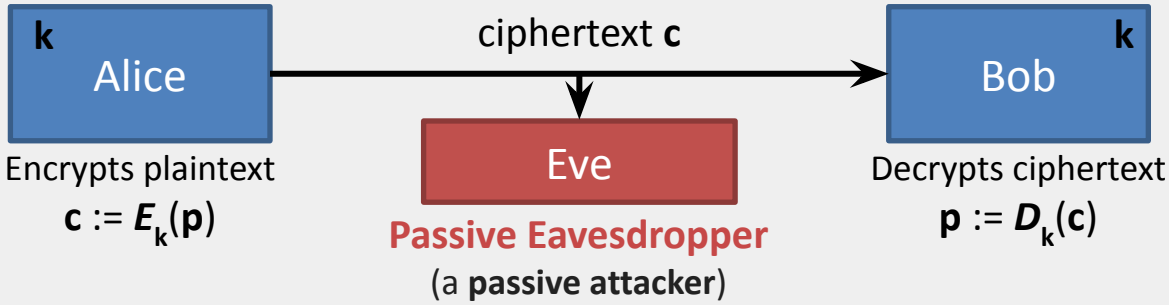
In practice: e.g., **HMAC-SHA-256**



Confidentiality (~~eavesdropping~~)

Construct ***E***() and ***D***() from secure PRG (a stream cipher) ***or*** secure PRP (a block cipher) with appropriate padding/cipher mode.

In practice: e.g., **AES-128 in CTR mode**



Today's lecture:

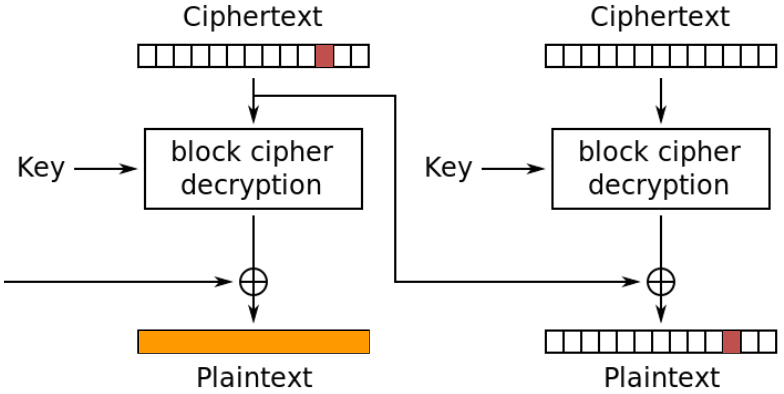
What if we want integrity and confidentiality *at the same time*?

Ciphertext Malleability

Caution: Many encryption methods are **malleable**: can *transform* a ciphertext into another ciphertext that decrypts to a related plaintext, without knowing the plaintext

Examples:

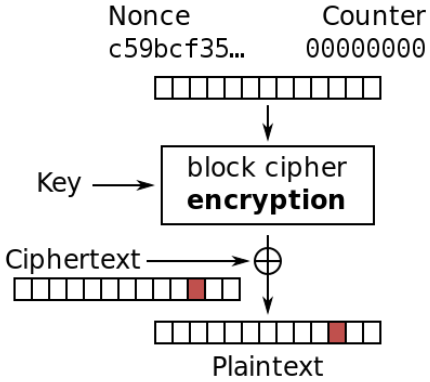
CBC mode decryption



Flipping bits in ciphertext block **i** will:

- completely corrupt decrypted block **i**
- flip corresponding bits in decrypted block **i**+1

Counter mode decryption



Flipping bits anywhere in the ciphertext will flip corresponding bits in decrypted plaintext

Need to use other methods to ensure integrity…

Authenticated Encryption (AE)

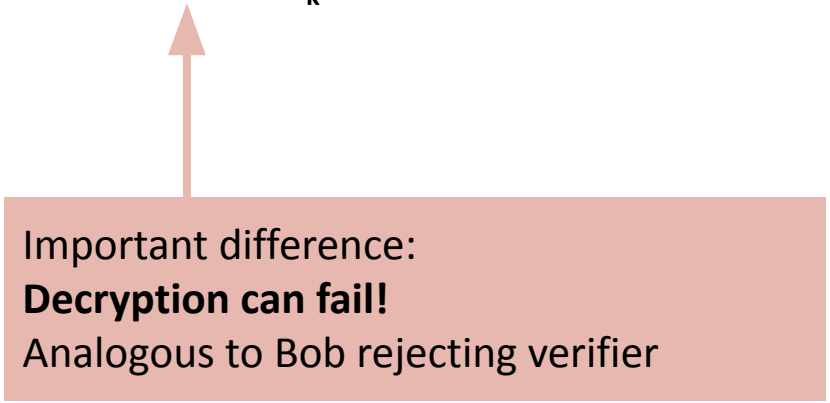
Two approaches:

1. Generically compose encryption and MAC
2. Build "all-in-one" primitive that does both

Syntax of AE:

$\mathbf{c} := E_k(\mathbf{p})$

$\mathbf{p}/\text{"fail"} := D_k(\mathbf{c})$



Security definition:

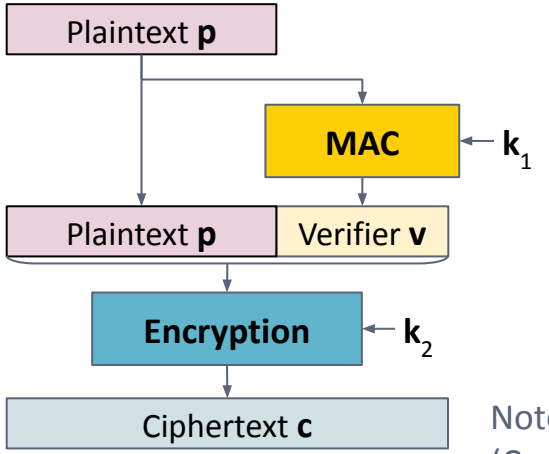
1. Let **k** be a secret key
2. Toss a coin (in secret) to get bit **b**
3. If **b**=0: $G() := E_k()$; $H() := D^*_k()$
   * **Modified to return "fail" on previous $E()$ outputs**
   If **b**=1: $G()$:= random bits; $H()$:= "fail"
4. Give Mallory $G()/H()$ oracles
   (Mallory gets to repeatedly query them)
5. Mallory guesses **b** *in polynomial time*

We say AE is **secure** if Mallory can't do meaningfully better than random guessing.

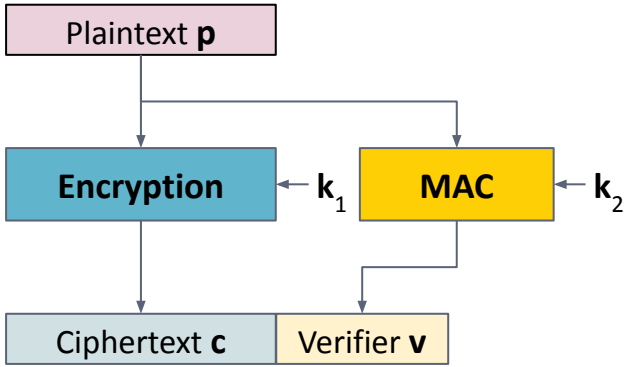
Composing Integrity and Confidentiality

How to ***compose*** our integrity and confidentiality protocols to achieve both? Three candidates:

MAC-then-Encrypt

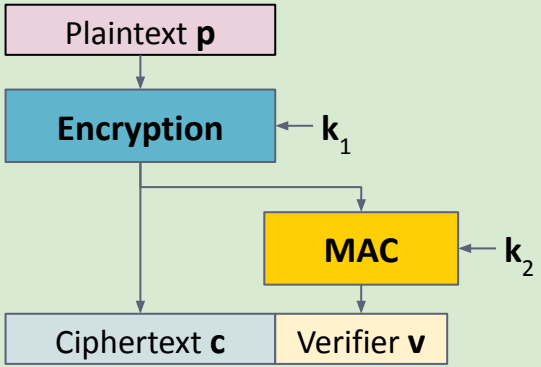


Encrypt-and-MAC



Note: **Use separate keys** for different purposes. (Can derive from a single key using PRG.) [Why?]

Encrypt-then-MAC



Safest approach

[Which approach is safest?]

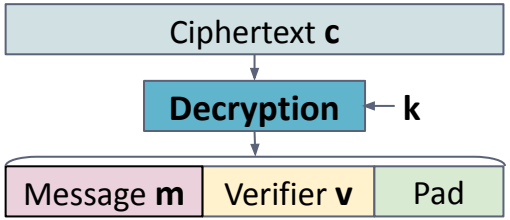
Our encryption methods (so far) only secure against passive eavesdroppers. Only **encrypt-then-MAC** ensures ciphertext isn't altered before decryption.

Example: CBC Padding Oracles

Common flaw when using **MAC-then-Encrypt**

Suppose an implementation uses **CBC mode**. Decryption involves the following steps:

1. **m**‖**v**‖**pad** := $D_k(\mathbf{c})$
2. Check that **pad** is valid **PKCS7**, else raise PadError
3. Check that **v** = $\mathbf{MAC}_k(\mathbf{m})$, else raise MacError



This is how TLS 1.0 worked. Seems reasonable?

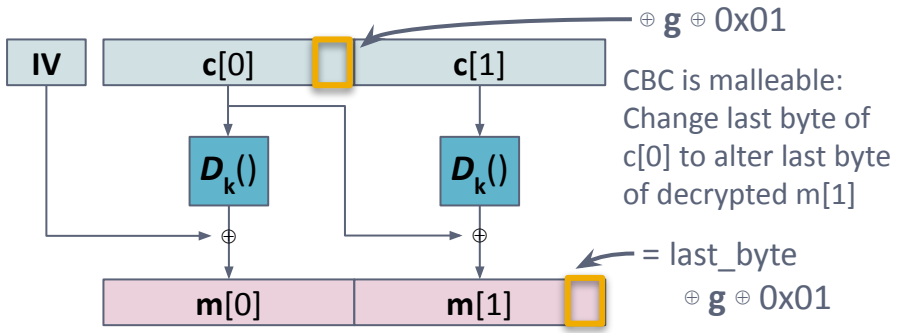
Any method to distinguish these two error types (even tiny timing differences) **leaks the plaintext!**

Padding oracle: attacker submits any ciphertext and learns if last bytes of plaintext are a valid pad

Example of a **chosen ciphertext attack**

Suppose attacker intercepts **c**, wants to learn **m.**

Step 1: Let **g** be a guess for last byte of block **m**[1]:



Step 2: Send modified ciphertext to padding oracle

If **g** = last_byte: **g** ⊕ last_byte ⊕ 0x01 = 0x01.
Modified plaintext ends in 0x01, so padding's valid; oracle returns MacError
else: Padding is invalid*, oracle returns PadError
(*Except for edge cases: e.g., what if m[1] ends in 0x02 0x01 and g = 0x02?)

Step 3: Repeat with **g** = 0,1, … 255 to learn last_byte. Then use a 0x02, 0x02 pad to learn next byte, etc.

Lesson: Encrypt *then* MAC You'll exploit in P1!

Authenticated Encryption with Associated Data

Preferred modern approach:

Authenticated encryption with associated data (AEAD)

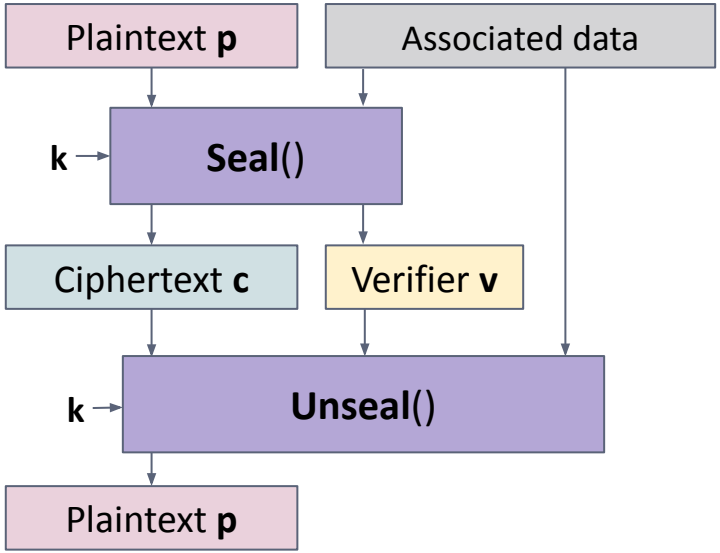
Integrity and encryption in a single primitive:

c, **v** := **Seal**(**k**, **p**, [***associated_data***])
encrypts plaintext **p** and returns
ciphertext **c** and a verifier **v** (called a "**tag**")

p, **err** := **Unseal**(**k**, **c**, **v**, [***associated_data***])
returns **p** or an error if **v** does not match the
supplied **c** and **associated_data**

Optional **associated_data**:

- Covered by verifier but **not encrypted**.
- Useful for binding data to its context:
  e.g., counter, sender ID, etc.



Examples:

AES-GCM ("Galois Counter Mode")
hardware accelerated in recent CPUs

ChaCha20-Poly1305, common on mobile

Galois/Counter Mode (GCM)

Galois/Counter Mode (GCM)

Most widely used AEAD cipher mode

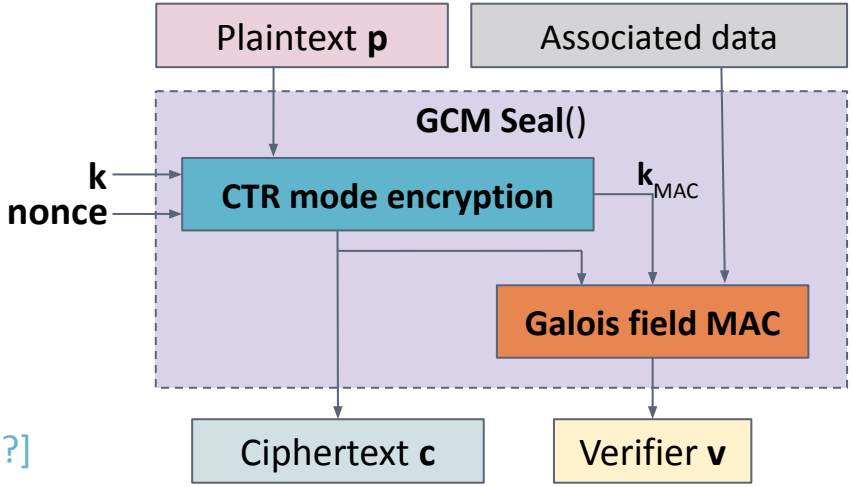
Developed by McGrew and Viega in 2004

Standardized by IETF, NIST, others

Non-generic composition [What kind?] of AES in CTR mode for encryption and a special MAC based on polynomials over finite ("Galois") fields.

Note: GCM violates principle of *key separation* [How?]

(We can prove it's ok, but it's delicate.)



Warning:

Can construct GCM ciphertexts that decrypt under many keys (differently, but without error)

Prof. Grubbs crafted one that decrypts under 131,072 different keys!

Warning: GCM nonce reuse is ***catastrophic***!

Encrypting two ciphertexts with the same (**k**,**nonce**) leaks the **plaintext *and the MAC key.***

Why? Polynomial root-finding.

Details interesting but beyond our scope in 388.

Parameter Sizes

Issue: How should we set sizes?

Choose $|\mathbf{k}|$ to resist brute force attacks, even as computers become faster.

For ciphers/PRG keys:

- Want to resist exhaustive search for **k**
- 128 bits considered "classically" safe
  ($2^{128} \approx 10^{38} \approx$ number of silicon atoms in the earth)
- For quantum-resistance, use 256 bits
  (Grover's algorithm gets attacker "sqrt" speedup)

For hash function outputs:

- Want collision resistance (CR)
- Need 2**n** bits of output for **n** bits of CR, due to "birthday" attacks
  (e.g., SHA-256 has 128 bits of CR)

Estimating what's feasible to compute?

$2^{64} \approx 10^{19}$ $2^{128} \approx 10^{38}$ 1 year $\approx 3 \times 10^7$ s

CPU mining: $\approx 10^8$ SHA-256/s
GPU mining: $\approx 10^{11}$ SHA-256/s
ASIC mining: $\approx 10^{14}$ SHA-256/s

Bitcoin miners globally: $\approx 10^{21}$ SHA-256/s

"Birthday" Attack

Generate random values, look for collision

Requires $2^{|\mathbf{k}|/2}$ time, $2^{|\mathbf{k}|/2}$ space

[Puzzle: Do it in constant space?]

For 128-bit output, takes 2^{64} steps: **doable!**

Coming Up

Reminders:

Project 1, Part 1 due Thursday at 6 p.m.
Project 1, Part 2 due 9/24 at 6 p.m.

Quiz on Canvas after every lecture

Thursday

Public Key Cryptography

Diffie-Hellman key exchange,
RSA encryption,
digital signatures

Starting Next Week

Web and Network Security Units